

# Plagiarism Detector — Architecture and Design

Project II · Gandaki College of Engineering and Science Supervisor: Er. Prativa Nyaupane

This document explains how the system is built and how it runs: the architecture, the design of each module, how the model works, how the frontend and the backend are connected, what happens on a single request, and how the demo is performed.

## Who studies what

The split follows the Division of Work recorded in the report and in the minutes. Every section title below carries its owner. **Everyone reads sections 1, 7, 8 and 9** — the panel can ask any of the three of us about the system as a whole.

| Member         | Role        | Owns in this document                  | Because they built                                                                                                                                    |
|----------------|-------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| Neon Neupane   | Team Leader | 2, 3, 4.1, 4.2, 5.4, 6.1, 6.2, 6.4     | Django and the three endpoints, extraction and OCR, preprocessing and Devanagari, the batching and sentence cache, error handling, the service wiring |
| Bishal Acharya | Member (M1) | 4.3, 4.4, 4.5, 5.1, 5.2, 5.3, 5.5, 5.6 | Sentence-BERT selection and fine-tuning, the TF-IDF baseline in PyTorch, both engines, the web scraping check, the evaluation                         |
| Nabin Giri     | Member (M2) | 6.2, 6.3, 6.5                          | The whole React and Vite frontend, uploader, matrix, highlight views, web check view, engine and threshold controls, responsive layout                |
| All three      | —           | 1, 7, 8, 9                             | The dataset work: corpus comparison, QQP and MRPC, the                                                                                                |

|  |  |  |                                                       |
|--|--|--|-------------------------------------------------------|
|  |  |  | sample documents,<br>the Nepali text and the<br>scans |
|--|--|--|-------------------------------------------------------|

---

## 1. What the system does · studied by All three

The system answers two questions about a document.

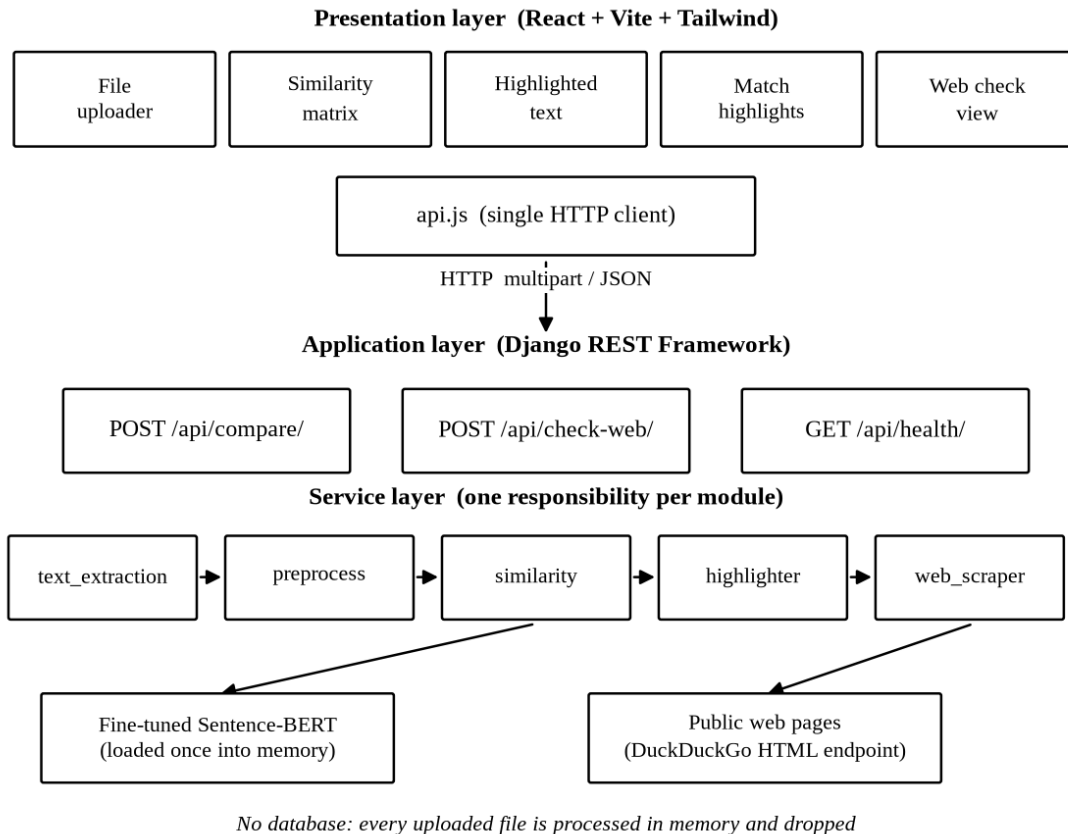
1. **Did these documents copy from each other?** A batch of documents is uploaded, every document is compared against every other one, and the result is a similarity matrix plus the list of sentences that match.
2. **Was this document copied from the web?** One document is uploaded, the system searches the web for it, opens the result pages and compares the document against them.

The point of the project is that the comparison is on **meaning**, not on words. Changing a few words defeats a word-matching checker; it does not defeat a sentence embedding.

Nothing is stored. A file is read in memory, used for the comparison, and dropped when the response is sent. The database is configured by Django but the application never writes to it.

---

## 2. System architecture · studied by Neon



System architecture

Three layers, with one direction of dependency — the browser talks to the API, the API talks to the services, the services talk to the model and to the web.

| Layer        | Technology                                          | Responsibility                                                |
|--------------|-----------------------------------------------------|---------------------------------------------------------------|
| Presentation | React 18, Vite, Tailwind CSS                        | Upload, settings, matrix, highlights                          |
| API          | Django, Django REST Framework                       | Parse the request, call a service, return JSON                |
| Service      | Python modules under <code>detector/services</code> | Extraction, preprocessing, similarity, highlighting, scraping |
| Model        | Sentence-BERT (PyTorch), TF-IDF (PyTorch)           | Turn text into vectors and compare them                       |

## 2.1 Why the layers are separated

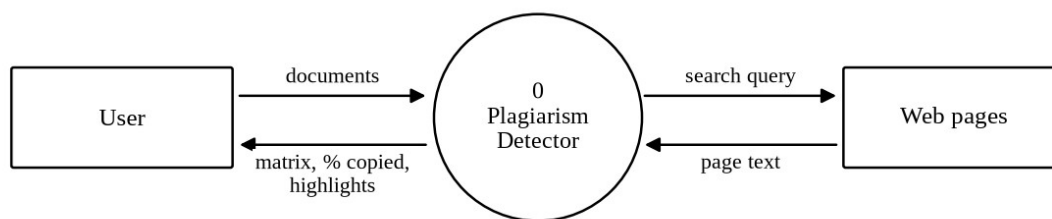
The views are deliberately thin. `views.py` opens with the line `"""HTTP layer -- thin. Views only parse the request, call a service, return JSON."""` No similarity logic lives in a view, so every service can be tested without starting Django, which is what the test files under `backend/tests/` do.

Each service is one file with one responsibility, so a new file format or a new engine is a change in one place.

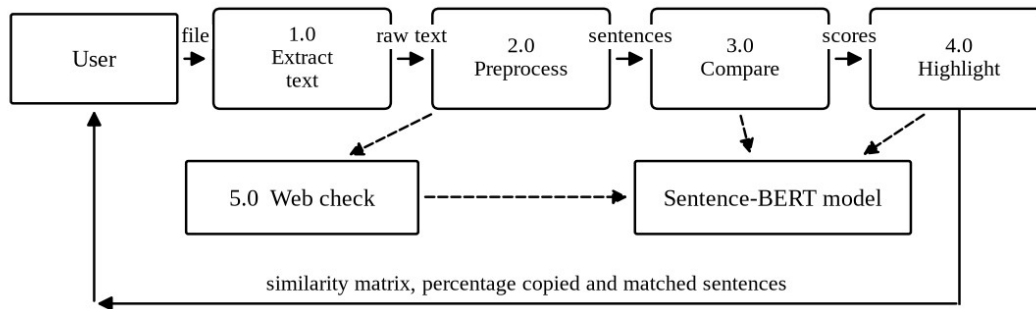
## 2.2 Directory layout

```
backend/  
  config/           Django project: settings, urls, wsgi  
  detector/  
    views.py        CompareView, WebCheckView, HealthView  
    urls.py          /api/compare/, /api/check-web/, /api/health/  
    services/  
      text_extraction.py  file bytes -> plain text  
      preprocess.py       text      -> clean text, tokens, sentences, keywords  
      similarity.py       texts     -> n x n similarity matrix  
      highlighter.py      two texts -> matched sentences + % copied  
      web_scraper.py      text      -> scored web sources  
    models/plagiarism-sbert/ the fine-tuned checkpoint (created by train.py)  
  tests/            unit tests for extraction, preprocessing, similarity  
frontend/  
  src/api.js        the only file that talks to the backend  
  src/App.jsx       tabs, settings, result layout  
  src/components/   FileUploader, SimilarityMatrix, HighlightedText,  
                   MatchHighlights, WebCheck  
train.py           one command fine-tuning, saves the checkpoint
```

## 3. Data flow · studied by Neon



*Level 0 data flow*



Level 1 data flow

The level 1 diagram shows the chain a document passes through. Nothing is written to a data store at any step — this was the point the supervisor asked to make explicit in the diagram.

upload -> extract -> preprocess -> embed -> compare -> highlight -> JSON response

## 4. Module design · studied by Neon and Bishal

### 4.1 Text extraction — `text_extraction.py` · studied by Neon

An uploaded file becomes plain text. The extension chooses the extractor from a lookup table, so adding a format is one function plus one dictionary entry.

| Type  | Extensions                                                                                          | How it is read                                                                     |
|-------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Text  | <code>.txt</code> , <code>.md</code>                                                                | Decoded as UTF-8, errors ignored                                                   |
| Word  | <code>.docx</code>                                                                                  | <code>python-docx</code> , paragraph text                                          |
| PDF   | <code>.pdf</code>                                                                                   | <code>pdfplumber</code> ; if no text layer, the pages are rendered and sent to OCR |
| Image | <code>.png</code> , <code>.jpg</code> , <code>.jpeg</code> , <code>.bmp</code> , <code>.tiff</code> | Tesseract OCR                                                                      |

Two design decisions matter here.

**A scanned PDF is detected, not rejected.** `_from_pdf` extracts the text layer; if the result is empty the file is a scan, so `_ocr_pdf` renders each page at 200 dpi and runs Tesseract over it.

**The OCR language is resolved once, from what is installed.** `ocr_lang()` asks Tesseract which languages it has. If `nep` is present it uses `eng+nep`, otherwise `eng`. The environment variable `OCR_LANG` overrides this. So the same code reads Nepali on a machine that has the Nepali data and does not crash on one that does not.

**Every failure is a message, never a 500.** Each extractor is wrapped, and any exception leaves through `ValueError` with a sentence meant for the user — `*"The file is empty."*`, `*"Unsupported file type ... Supported: ..."*`, `*"No text could be read. If this is a scan, use a clearer image."*` The view keeps the failed file in the list with an `error` key instead of dropping it, so the interface can say which upload failed and why.

## 4.2 Preprocessing — `preprocess.py` · studied by Neon

Four small functions, all pure.

- `clean(text)` — lowercase and collapse whitespace.
- `tokenize(text)` — words of two or more characters matching `[a-z0-9-ॐ-ॐ]`, with stopwords removed. The character class covers the Devanagari block, so Nepali words survive tokenisation.
- `split_sentences(text)` — splits on `.`, `!`, `?`, and on the Nepali danda `|` and double danda `||`. A danda written with no space after it is handled by a second split. Fragments of ten characters or fewer are dropped, because they add noise to highlighting.
- `keywords(text, k=6)` — the six most frequent content words of four or more characters. This builds the web search query.

The stopwords list carries English function words and Nepali ones (`र, मा, को, छ, हुन्छ, ...`) for the same reason: they are frequent, carry no meaning, and would dominate a TF-IDF vector.

## 4.3 Similarity — `similarity.py` · studied by Bishal

Two engines behind one function, `all_pairs_matrix(texts, method)`. Both are built on PyTorch tensors; scikit-learn is deliberately not used, so the vector arithmetic is visible in the code.

**TF-IDF engine (the baseline).** Builds a vocabulary from the token lists, computes a smoothed inverse document frequency

$$\text{idf}(t) = \log((1 + N) / (1 + \text{df}(t))) + 1$$

then for each document a term-frequency vector divided by the document length and multiplied by the idf. The matrix is the cosine similarity between every pair of rows. This engine cannot see a paraphrase — it only sees shared words — and it is in the project precisely to show that.

**Sentence-BERT engine (the real one).** Described in section 5.

Both end at the same helper, `_cosine_matrix`, which L2-normalises the rows and multiplies the matrix by its own transpose, clamped to `[-1, 1]` and rounded to four decimals.

#### 4.4 Highlighting — `highlighter.py` · studied by Bishal

`compare_documents(text_a, text_b, threshold)` is what turns a single number into something a person can check.

3. Split both documents into sentences.
4. Embed every sentence of A and every sentence of B.
5. Compute the full cosine matrix between them with `util.cos_sim`.
6. For each sentence of A take its best match in B. If the score is at or above the threshold the sentence is marked copied and the matching sentence of B is attached.
7.  $\text{percent\_copied} = 100 \times \text{copied sentences} / \text{total sentences of A}$ .

The percentage is therefore a count of sentences, not an average of scores, which is what makes it defensible in the viva: every point of the percentage can be pointed at on screen.

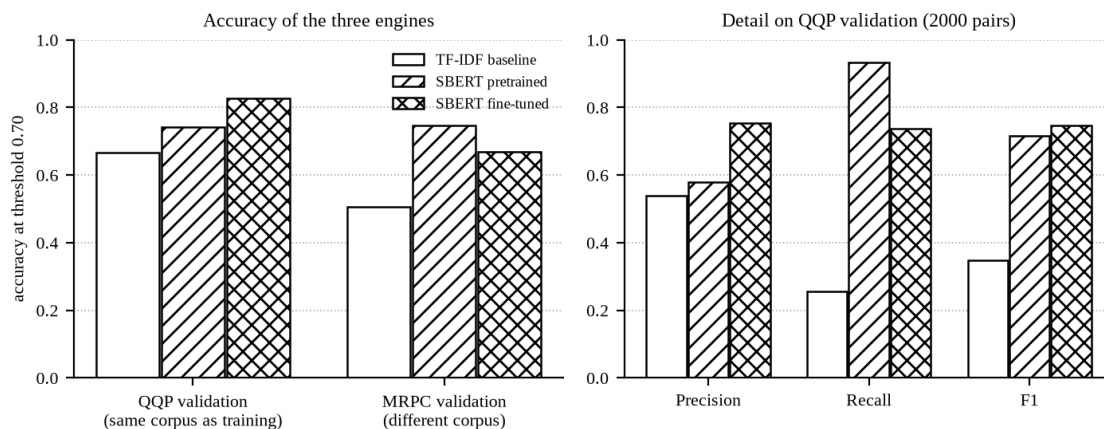
#### 4.5 Web checking — `web_scraper.py` · studied by Bishal

8. `keywords()` produces a six word query.
9. The query is posted to the DuckDuckGo HTML endpoint — no API key, no quota.
10. Up to ten result links are opened. `script`, `style`, `nav`, `header` and `footer` are removed, the `<p>` text is joined and cut to 5000 characters.
11. The page and the document are embedded and compared, and the sentence-level highlighting is run against the page.
12. **A page whose `percent_copied` is zero is discarded.** A high document score on its own is topical overlap, not plagiarism; only pages that share actual sentences are shown.
13. The surviving sources are sorted by score, highest first.

Every network call is inside a `try`, and a failure returns an empty result rather than an exception. A page that will not load is skipped and the request still answers. This is why the demo survives a bad network — the point raised in the sixth meeting.

---

## 5. How the model works · studied by Bishal



Engine comparison

## 5.1 The idea · studied by Bishal

A word-matching checker compares strings. `"The cat sat on the mat"` and `"The feline rested upon the rug"` share almost no words, so a string checker scores them near zero, although one is a rewrite of the other.

Sentence-BERT maps a sentence to a 384-dimensional vector, trained so that sentences with the same meaning land close together. The comparison is then the cosine of the angle between the two vectors:

$$\text{similarity}(a, b) = (a \cdot b) / (|a| \times |b|)$$

The value runs from -1 to 1. Above the threshold of **0.7**, the pair is flagged.

## 5.2 The base model and the fine-tuning · studied by Bishal

The base model is `all-MiniLM-L6-v2` — six transformer layers, 384-dimensional output, small enough to run on the CPU of a laptop.

`train.py` fine-tunes it on labelled paraphrase pairs:

| Setting    | Value                                                    |
|------------|----------------------------------------------------------|
| Dataset    | Quora Question Pairs (QQP), about 364,000 labelled pairs |
| Pairs used | 30,000 ( --sample)                                       |
| Epochs     | 1                                                        |
| Batch size | 32                                                       |
| Loss       | <code>CosineSimilarityLoss</code>                        |
| Warm-up    | 10% of the steps                                         |
| Held out   | 10% for evaluation                                       |
| Output     | <code>backend/detector/models/plagiarism-sbert</code>    |

`CosineSimilarityLoss` pushes the cosine of a labelled-similar pair towards 1 and of a labelled-different pair towards 0. That is exactly the quantity the detector uses at runtime, so the model is trained on the same measure it is judged by.

The backend picks the checkpoint up on its own. `SbertEngine.get_model()` loads `models/plagiarism-sbert` if the directory exists and falls back to the pretrained `all-MiniLM-L6-v2` otherwise, and `/api/health/` reports which of the two is live through `model_fine_tuned`. The interface shows it as **Fine-tuned** or **Pretrained** on a stat card.

## 5.3 From a document to one vector · studied by Bishal

A transformer has a fixed input window, so a long document does not fit. Feeding it whole silently truncates the tail.



`chunk_text()` splits the document into chunks of **five sentences**. Every chunk is embedded and the document vector is the **mean of its chunk vectors**. Nothing is dropped.

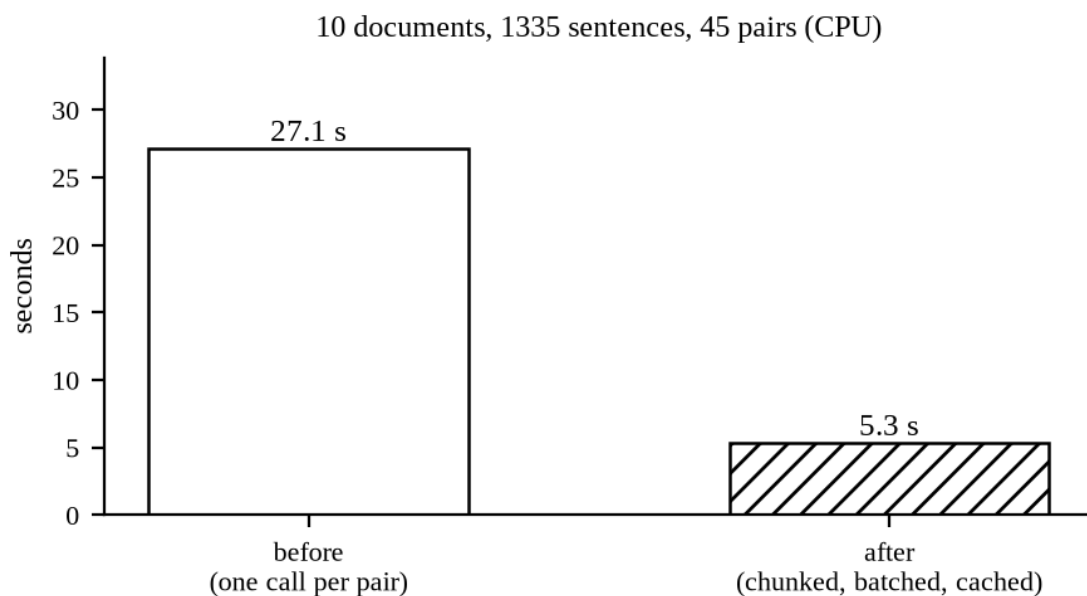
#### 5.4 The two speed optimisations · studied by Neon

The eighth meeting reported that a large batch was slow. Two changes fixed it, and both are visible in `similarity.py`.

**Batched encoding.** `document_embeddings()` flattens the chunks of *\*all\** documents into one list and makes a single `model.encode(..., batch_size=32)` call, then slices the result back per document. The model is entered once per request instead of once per document.

**A sentence cache.** The same sentence appears in many pairs of an  $n \times n$  matrix.

`SbertEngine._cache` maps a string to its vector, so a sentence is encoded once per request; `embed()` only sends the strings it has not seen. The cache is capped at 4096 entries and cleared when it would overflow.



Timing

| Measurement                                                         | Before | After |
|---------------------------------------------------------------------|--------|-------|
| 10 documents, 2628 words each, 1335 sentences, 45 highlighted pairs | 27.1 s | 5.3 s |

**A 5.1× speed-up on CPU**, with no change to the output.

#### 5.5 Measured accuracy · studied by Bishal

QQP, 2000 held-out pairs, 692 of them positive, threshold 0.7:

| Engine | Accuracy | Precision | Recall | F1 | Seconds |
|--------|----------|-----------|--------|----|---------|
|--------|----------|-----------|--------|----|---------|

|                         |              |              |       |              |      |
|-------------------------|--------------|--------------|-------|--------------|------|
| TF-IDF baseline         | 0.667        | 0.538        | 0.256 | 0.347        | 0.3  |
| SBERT pretrained        | 0.743        | 0.580        | 0.932 | 0.715        | 19.1 |
| <b>SBERT fine-tuned</b> | <b>0.826</b> | <b>0.754</b> | 0.737 | <b>0.746</b> | 6.1  |

The number that matters most is **separation** — the gap between the mean score of a plagiarised pair and of an unrelated pair:

| Engine                  | Mean positive | Mean negative | Separation   |
|-------------------------|---------------|---------------|--------------|
| TF-IDF                  | 0.537         | 0.334         | 0.203        |
| SBERT pretrained        | 0.866         | 0.556         | 0.310        |
| <b>SBERT fine-tuned</b> | 0.779         | 0.331         | <b>0.448</b> |

Fine-tuning more than doubles the separation of the baseline. The pretrained model scores \*everything\* high — mean 0.556 even on unrelated pairs — so a single threshold works badly for it. Fine-tuning pushes the unrelated pairs down to 0.331 and leaves the plagiarised ones at 0.779, which is what makes a fixed threshold of 0.7 usable.

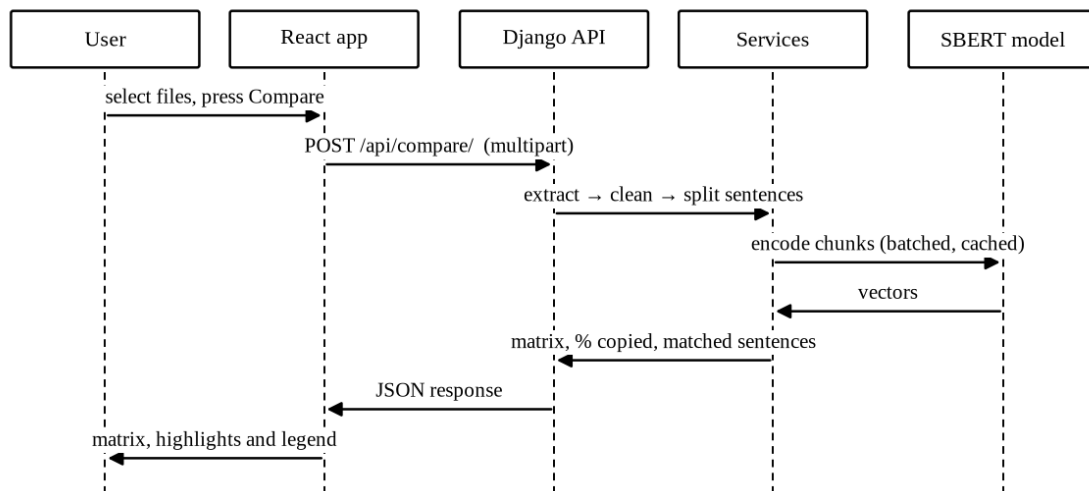
**Honest limitation.** On MRPC, a corpus the model was not tuned on, the fine-tuned model scores 0.669 accuracy against the pretrained model's 0.748. Tuning on QQP moved the model towards QQP's idea of a paraphrase. This is domain shift and it is reported rather than hidden.

## 5.6 The threshold · studied by Bishal

0.7 was fixed experimentally on the sample documents and it is the default, not a constant — the interface exposes a slider from 0.40 to 0.95 and the chosen value is sent with the request. The API clamps whatever arrives to **[0.1, 0.99]**.

---

## 6. Frontend and backend connection · studied by Nabin and Neon



Sequence

### 6.1 The contract · studied by Neon

Three endpoints, all under `/api/`.

| Method | Path                         | Purpose                                              |
|--------|------------------------------|------------------------------------------------------|
| GET    | <code>/api/health/</code>    | Liveness, and whether the fine-tuned model is loaded |
| POST   | <code>/api/compare/</code>   | Compare a batch of documents against each other      |
| POST   | <code>/api/check-web/</code> | Compare one document against web pages               |

There is no authentication — `DEFAULT_AUTHENTICATION_CLASSES` is empty and the permission class is `AllowAny`. The system stores nothing and has no accounts, so there is nothing to protect behind a login.

### 6.2 How the two servers meet · studied by Nabin and Neon

In development the two run separately: Vite on **5173**, Django on **8000**. They are joined in two independent ways.

**A Vite proxy.** `vite.config.js` forwards `/api` to `http://localhost:8000`, so the browser only ever calls its own origin and the request is same-origin.

```
server: { port: 5173, proxy: { '/api': 'http://localhost:8000' } }
```

**CORS headers.** `corsheaders.middleware.CorsMiddleware` runs first in the middleware list with `CORS_ALLOW_ALL_ORIGINS = True`, so the API also answers a direct cross-origin call — useful when the frontend is served from somewhere else, or when testing with `curl`.

Because `api.js` uses the relative base `'/api'`, the same build works in both situations.

### 6.3 One place talks to the backend · studied by Nabin

`frontend/src/api.js` is the only file in the frontend that knows the backend exists. It exports two functions and holds one `post()` helper that unwraps the error body, so a failure arrives in the interface as `detail` from the API rather than as a status code.

```
const BASE = '/api'

export async function compareDocuments(files, method = 'sbert', threshold) {
  const form = new FormData()
  for (const file of files) form.append('files', file)
  form.append('method', method)
  if (threshold != null) form.append('threshold', threshold)
  return post('/compare/', form, true)
}
```

Files go as `multipart/form-data`; pasted text goes as JSON. The backend accepts both, because `CompareView` and `WebCheckView` declare `parser_classes = [MultiPartParser, FormParser, JSONParser]` and `_collect_documents()` reads uploads from `request.FILES` and pasted items from `request.data['texts']` into the same `[{name, text}]` list.

### 6.4 Request and response · studied by Neon

**Request** — `POST /api/compare/`, `multipart: files` (repeated), `method (sbert | tfidf)`, `threshold` (a number).

**Response** — `200 OK`:

```
{
  "documents": ["doc1_original.txt", "doc2_paraphrased.txt"],
  "method": "sbert",
  "model_fine_tuned": true,
  "threshold": 0.7,
  "matrix": [[1.0, 0.6676], [0.6676, 1.0]],
  "flagged_pairs": [
    {
      "doc_a": "doc1_original.txt",
      "doc_b": "doc2_paraphrased.txt",
      "score": 0.6676,
      "percent_copied": 25.0,
      "matches": [
        { "sentence": "...", "matched_with": "...", "score": 0.81, "copied": true }
      ]
    }
  ],
  "skipped": [{ "name": "broken.pdf", "error": "The file could not be read (PdfError)."}]
}
```

`skipped` is the field that keeps a bad upload visible. The comparison runs on the readable documents and the interface prints an amber panel naming the files that were left out.

**Errors** — `400` with a `detail` string: fewer than two readable documents, or an unknown `method`.

## 6.5 Which component consumes what · studied by Nabin

| Response field                 | Component                                                              | What it draws                                 |
|--------------------------------|------------------------------------------------------------------------|-----------------------------------------------|
| <code>documents, matrix</code> | <code>SimilarityMatrix.jsx</code>                                      | The $n \times n$ table, coloured by band      |
| <code>flagged_pairs</code>     | <code>HighlightedText.jsx</code> ,<br><code>MatchHighlights.jsx</code> | Sentence pairs, copied ones marked            |
| <code>sources</code>           | <code>WebCheck.jsx</code>                                              | Each page with its link, score and percentage |
| <code>skipped</code>           | <code>App.jsx</code>                                                   | The amber "some files were skipped" panel     |
| <code>model_fine_tuned</code>  | <code>App.jsx</code>                                                   | The <b>Fine-tuned / Pretrained</b> stat card  |
| <code>threshold</code>         | <code>App.jsx</code>                                                   | The score legend bands                        |

`App.jsx` holds the state — `files`, `result`, `loading`, `error`, `method`, `threshold` — and passes it down. There is no state library; the tree is shallow enough that `useState` in one component is enough.

---

## 7. What happens on one request · studied by All three

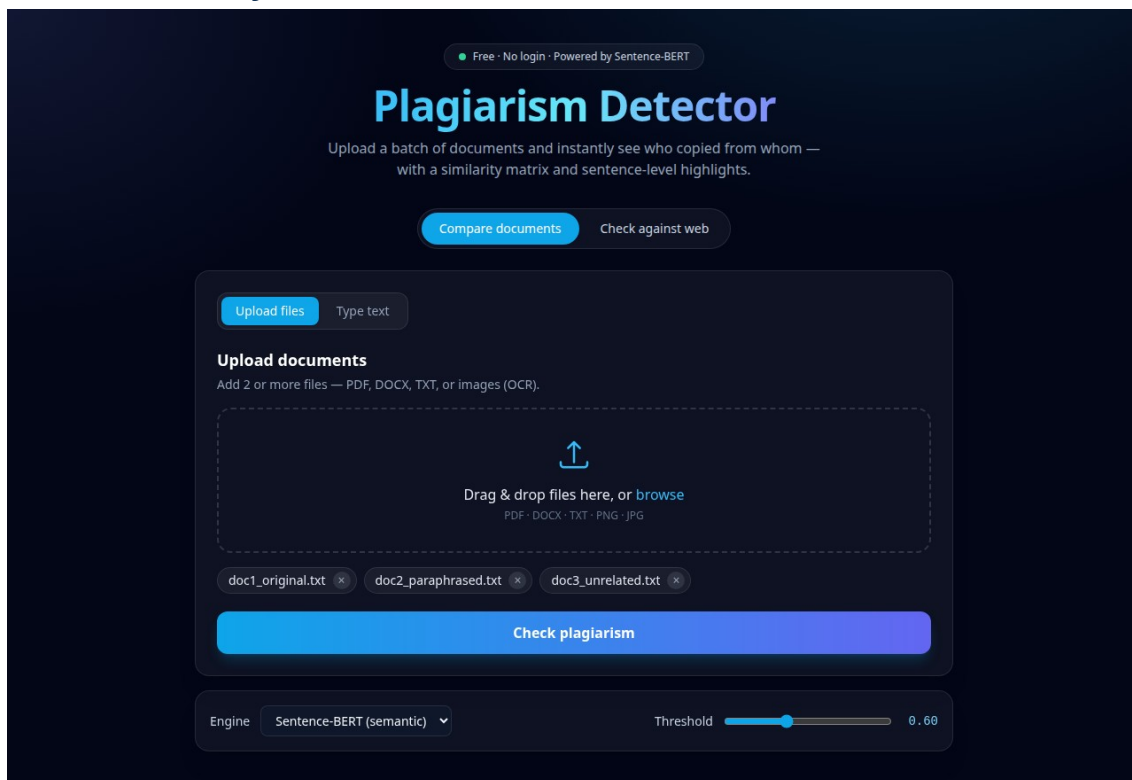
A batch comparison, from the click to the screen.

- Browser.** Files are dropped on `FileUploader`. The engine and the threshold are read from `Settings.handleCompare()` sets `loading` and calls `compareDocuments()`.
- Transport.** `api.js` builds a `FormData` and posts it to `/api/compare/`. Vite forwards it to Django on port 8000.
- View.** `CompareView.post()` runs `_collect_documents()`. Each file goes to `extract_text()`; a failure is recorded in `skipped` instead of raising.
- Guard.** Fewer than two readable documents, or a `method` that is not `sbert` or `tfidf`, returns `400` with a `detail` message.
- Matrix.** `similarity.all_pairs_matrix(texts, method)`. For SBERT: chunk every document into groups of five sentences, embed all chunks of all documents in one batched call, average the chunks per document, cosine-compare every pair.

19. **Highlights.** For every pair at or above the threshold, `highlighter.compare_documents()` embeds the sentences of both documents — hitting the cache filled in step 5 — takes the best match for each sentence of A, and computes `percent_copied`.
20. **Response.** The view returns the matrix, the flagged pairs, the threshold, the skipped files and whether the fine-tuned model is loaded.
21. **Render.** `App.jsx` stores it. `SimilarityMatrix` draws the table, `HighlightedText` draws the sentence pairs, the stat cards show the counts. The uploaded bytes are already gone from the server.

---

## 8. Demo · studied by All three



*Upload*

### 8.1 Starting the system

Two terminals.

```
# terminal 1 – backend
cd backend
python manage.py runserver          # http://localhost:8000

# terminal 2 – frontend
cd frontend
npm run dev                         # http://localhost:5173
```

Check the backend before demonstrating anything:

```
curl http://localhost:8000/api/health/  
# {"status": "ok", "model_fine_tuned": true}
```

`model_fine_tuned: true` means the checkpoint is loaded. If it is `false`, `train.py` has not been run and the demo will use the pretrained model — the numbers below will differ.

## 8.2 The sample documents

`samples/` holds four documents chosen so that each one proves a different point.

| Document                          | Purpose                                                       |
|-----------------------------------|---------------------------------------------------------------|
| <code>doc1_original.txt</code>    | The source                                                    |
| <code>doc2_paraphrased.txt</code> | The same content, rewritten — the case the project exists for |
| <code>doc3_unrelated.txt</code>   | A different topic — must score near zero                      |
| <code>doc4_nepali.txt</code>      | Devanagari — proves the Nepali path                           |

## 8.3 The demonstration

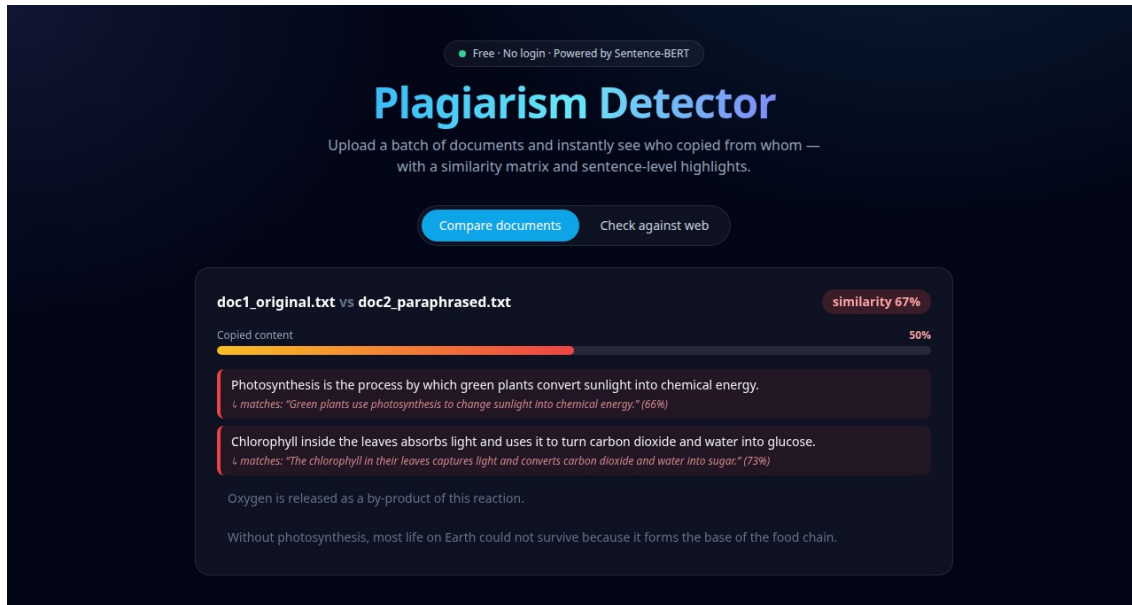
**Step 1 — show the baseline fail.** Upload all four documents, set the engine to **TF-IDF**, compare.



*Similarity matrix*

`doc1` against `doc2` scores **0.468** — below the 0.7 threshold, so the paraphrase is *\*not\** flagged. This is the failure the project is about. Say it out loud before fixing it.

**Step 2 — show the model succeed.** Switch the engine to **Sentence-BERT** and compare the same four files. **doc1** against **doc2** rises to **0.668**, and the pair reports **25% copied — 1 of 4 sentences**, with the matched sentence shown next to its source. The unrelated pair stays at **0.006** and the Nepali pair at **-0.018**.



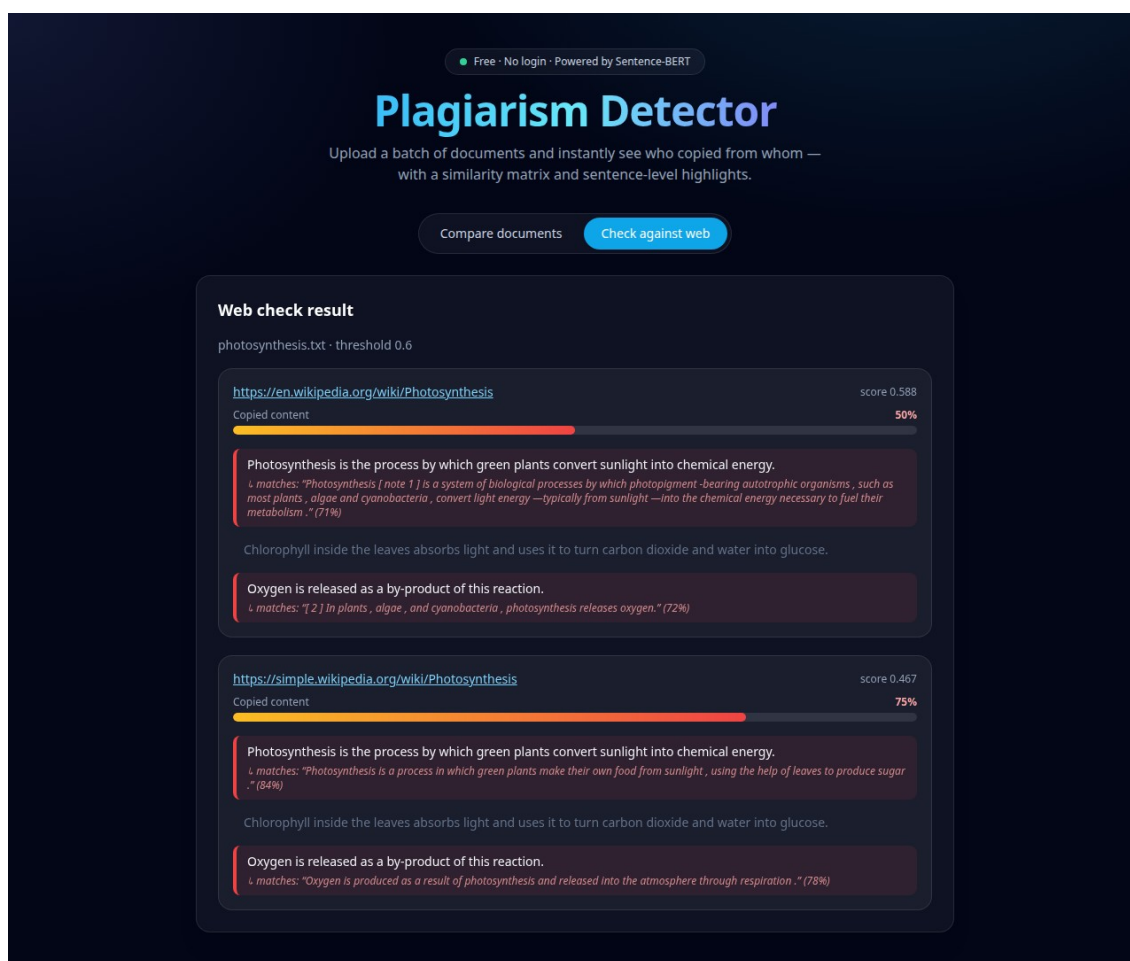
### Highlights

Point at the sentence-level panel: the percentage is a count of matched sentences, so every claim on the screen can be checked by eye.

**Step 3 — move the threshold.** Drag the slider from 0.70 down to 0.65. The paraphrased pair is now flagged. This shows the threshold is a policy choice, not a magic constant, and that the value travels with the request.

**Step 4 — the web check.** Open the **Check against web** tab, paste a paragraph from a well known page, run it.





#### Web check

Each source comes back with its link, its score and its percentage, sorted highest first. Mention that pages sharing no sentence are dropped even when their topic score is high.

**Step 5 — Nepali.** Upload `doc4_nepali.txt` with a second Nepali document. The danda is treated as a sentence end, so the highlighting works on Devanagari as well.

**Step 6 — failure handling.** Upload a corrupt or empty file together with two good ones. The comparison still runs and an amber panel names the bad file with its reason. Nothing 500s.

### 8.4 If something goes wrong

| Symptom                              | Cause                                    | Fix                                                           |
|--------------------------------------|------------------------------------------|---------------------------------------------------------------|
| <code>model_fine_tuned: false</code> | The checkpoint is missing                | <code>python train.py --sample 30000 --epochs 1</code>        |
| The web check returns nothing        | The network or DuckDuckGo is blocked     | Pass explicit URLs — the API accepts a <code>urls</code> list |
| A scan returns no text               | Tesseract is missing or the scan is poor | Install Tesseract; use a clearer image                        |
| Nepali OCR reads gibberish           | The <code>nep</code> traineddata is not  | Install it, or set <code>OCR_LANG=eng</code>                  |

|                              |                      |                                               |
|------------------------------|----------------------|-----------------------------------------------|
|                              | installed            |                                               |
| The first comparison is slow | The model is loading | It is a singleton; the second request is fast |

---

## 9. Limitations and future work · studied by All three

### Limitations.

- Accuracy on Nepali is lower than on English; the model was fine-tuned on English pairs.
- The fine-tuned model loses to the pretrained one on MRPC — domain shift from QQP.
- Sentence splitting is regex-based; an abbreviation such as `*"Dr."*` can split a sentence early.
- The web check depends on one search endpoint with no API key, so it is rate-limit bound.
- Everything runs in one process, in memory; there is no queue, so a very large batch ties up the request.

### Future work.

- Fine-tune on a Nepali paraphrase corpus.
- Cache document embeddings between requests, not only within one.
- Move the comparison to a task queue and stream progress to the browser.
- Add a report export (PDF) of the matrix and the highlights.